QUALITY ASSURANCE / ISTQB v4.0

TEST PLAN

API & AUTOMATION SCOPE

Pickleball Court & Coach Booking System (PCS)

Document Control	Value
Version	v1.2
Prepared by	LÊ THỊ VĂN ANH - QA Leader
Reviewed by	TRƯƠNG QUANG TUÂN - Tester 1
Approved by	TRẦN QUỐC SANG - Project Manager
Date	19/07/2026
Status	Review - current execution blockers recorded

Current verification: 35 tests executed and passed; 7 suites failed during collection because required packages were unavailable from the root test workspace. Exit criteria are therefore not yet met.

Internal project document

Document information

Field	Value
Project Name	Pickleball Court & Coach Booking System (PCS)
Document Title	TEST PLAN - API & AUTOMATION SCOPE
Version	v1.2
Prepared By	LÊ THỊ VĂN ANH - QA Leader
Reviewed By	TRƯƠNG QUANG TUÂN - Tester 1
Approved By	TRẦN QUỐC SANG - Project Manager
Date	19/07/2026
Status	Review

Revision history

Version	Date	Author	Description
1.0	10/10/2023	LÊ THỊ VĂN ANH	Initial API and automation test plan.
1.1	19/07/2026	LÊ THỊ VĂN ANH	Updated Vitest workspace and QA Dashboard scope.
1.2	19/07/2026	LÊ THỊ VĂN ANH	Reconciled technology versions and recorded current execution blockers.

Contents

1. INTRODUCTION

This Test Plan defines the approach, scope, schedule, resources, controls, and completion criteria for API integration and automation testing of PCS. It provides a shared basis for planning, execution, defect triage, reporting, and release decisions.

Testing protects the core business flow - account access, court and coach booking, promotion validation, payment confirmation, and operational reporting - while reducing regression risk across Next.js route handlers, service modules, SQL Server interactions, and external integrations.

Objective: Provide repeatable, risk-based evidence that critical PCS workflows behave correctly and that known blockers are visible before release approval.

2. PROJECT OVERVIEW

PCS supports online court and coach reservations, player matching, payment processing, promotions, reviews, notifications, administrative reporting, and AI-assisted customer interaction. The solution uses separate frontend and backend Next.js applications plus a FastAPI-based AI service and a React/Vite QA dashboard.

Area	Verified repository baseline
Frontend	Next.js 15.x, React 18.3, TypeScript
Backend	Next.js 16.2, React 19.2, Node.js route handlers
Database	Microsoft SQL Server through mssql; repositories mocked in automated tests
Authentication	JWT, bcrypt/bcryptjs, Google OAuth integration
Payments	PayOS SDK/webhook routes; refund helper integrations
AI service	FastAPI/Python service for chatbot and player/coach scoring
QA dashboard	Vite/React dashboard generated from test and coverage artifacts

3. TEST OBJECTIVES

- Verify critical authentication, court, booking, coach, payment, promotion, review, notification, reporting, matching, and AI rules.
- Validate API status codes, authorization gates, input validation, repository interactions, and external callback handling.
- Detect overlap, boundary, timezone, voucher, refund, and security defects early.
- Maintain a repeatable regression suite and publish evidence through JSON/HTML coverage and the PCS QA Dashboard.

- Target at least 90% statement coverage and 85% branch coverage for explicitly targeted business-logic modules.

4. TEST SCOPE

4.1 In scope

Module	Coverage focus
Authentication	Registration/login validation, duplicate checks, JWT issuance and access control
Court & Booking	Court availability, time-slot overlap, daily limits, booking lifecycle and holds
Coach	Coach profiles, approval status and appointment slots
Payment & Refund	Checkout creation, PayOS webhook handling, status changes and refund rules
Promotion	Active dates, minimum order thresholds and invalid voucher handling
Other services	Reviews, notifications, reports, player matching and AI fallbacks
Automation assets	Vitest workspace/configuration, shared test data, mocks, coverage output and dashboard scripts

4.2 Out of scope

- High-volume load, stress and endurance testing.
- Production payment execution and production customer data.
- Comprehensive mobile-device and cross-browser manual certification.
- Infrastructure penetration testing and third-party vendor certification.
- Formal UAT execution by customer stakeholders.

5. TEST ITEMS

Item	Repository location / description
API test suites	tests/api/auth.api.test.ts, booking.api.test.ts, court.api.test.ts, payment.api.test.ts
Unit suites	tests/unit/*.test.ts covering 12 service-oriented areas
UI suite	frontend/tests/ui/login.ui.test.tsx
Shared data and mocks	tests/data/testData.ts; tests/mock/*.ts; setupBackendTests.ts; setupTests.ts
Payment route	backend/src/app/api/payments/payos-webhook/route.ts
Workspace configuration	vitest.workspace.ts and vitest.config.ts
Dashboard	scripts/generate-dashboard-data.ts, scripts/serve-dashboard.js, test-dashboard-src/

6. TEST TYPES

Test type	Purpose
Unit testing	Isolate service/business rules using mocked repositories and dependencies.
API integration testing	Exercise route-handler behavior with request and repository mocks.
UI component testing	Validate login form behavior in JSDOM with React Testing Library.
Regression testing	Re-run automated suites after changes to detect unintended impact.
Security-focused testing	Check authentication, authorization, validation and webhook trust boundaries.
Coverage analysis	Measure statement, branch, function and line execution with V8 coverage.

7. TEST APPROACH / TEST STRATEGY

Testing is automated-first, risk-based, and aligned with the repository's Vitest workspace. Backend suites run in a Node environment; frontend component tests run in JSDOM. Database and third-party behavior is isolated with mocks so that failures can be reproduced deterministically.

Level	Approach	Primary evidence
Unit	White-box branch and error-path validation	Vitest unit suite results
API integration	Request/response and service interaction checks	API suite results and status assertions
UI component	Black-box validation of login interactions	React Testing Library assertions
Regression	Full workspace execution after material changes	JSON test report and dashboard
Coverage	V8 instrumentation with excluded build/test artifacts	HTML/JSON coverage reports

Test design techniques

- Equivalence partitioning for valid/invalid credentials and promotion eligibility.
- Boundary value analysis for time slots, daily booking limits, rating ranges and discount thresholds.
- Decision tables for compound registration validation and payment state rules.
- State transition testing for booking/payment lifecycle changes.
- Use-case testing for book-court-and-pay end-to-end behavior.

8. ENTRY CRITERIA

- Requirements and acceptance criteria for the target module are available and reviewed.

- Target route/service implementation compiles and is reachable by the test workspace.
- Root and application dependencies required by imported modules are installed.
- Shared mocks and deterministic test data are prepared.
- Test environment variables contain only sandbox/non-production credentials.
- The build under test is identified and deployed to the intended test environment.

9. EXIT CRITERIA

Criterion	Target	Current assessment (19/07/2026)
Planned tests executed	100%	Not met - 7 suites failed during collection
Executed tests pass	100%	Met for executed set - 35/35 passed
Critical/high defects	0 open	Pending triage of collection blockers
Targeted statement coverage	>= 90%	Baseline report states 92.50%; rerun required after blockers
Targeted branch coverage	>= 85%	Baseline report states 88.75%; rerun required after blockers
Dashboard and reports	Generated and reviewable	Artifacts exist; refresh after successful full run

Release gate: The document remains in Review status until all suites collect and execute successfully, results are refreshed, and blocking defects are closed or formally accepted.

10. TEST ENVIRONMENT

Component	Configuration
Operating system	Windows 10/11 development environment
Runtime	Node.js; Python/FastAPI for AI service
Test framework	Vitest 3.0.7 declared at repository root
Backend test environment	Node with backend aliases and setupBackendTests.ts
Frontend test environment	JSDOM with setupTests.ts
Coverage provider	@vitest/coverage-v8 3.0.7
Database	Mocked Microsoft SQL Server connections
External services	Mocked PayOS, email/OAuth, refund and AI dependencies

11. TEST TOOLS

Tool	Purpose
Vitest	Unit, API integration, UI and regression execution
React Testing Library	DOM rendering and interaction assertions
V8 coverage	Statement, branch, function and line metrics
TypeScript / tsx	Typed test and dashboard scripts
Postman assets	Manual API collection/environment support
Vite + React dashboard	Visual test, traceability, coverage and defect reporting
Git / npm scripts	Version control and repeatable execution commands

12. TEST DATA MANAGEMENT

Reusable payloads are centralized in tests/data/testData.ts and module-specific mocks in tests/mock/. Tests must reset mock state between cases, avoid real customer data, and use deterministic timestamps and identifiers where business rules are time-sensitive.

Dataset	Purpose	Control
Valid/invalid auth	Token and validation paths	Synthetic identities only
Overlapping slots	Booking-conflict detection	Fixed court/date/time records
Voucher boundaries	Expiry and order thresholds	Dates relative to controlled clock
Payment callbacks	Signature and status transitions	Sandbox/mock payloads
Large/edge records	Boundary and resilience paths	Generated, non-production data

13. ROLES AND RESPONSIBILITIES

Role / person	Responsibilities
Project Manager - Trần Quốc Sang	Approve scope and deliverables; coordinate use cases and decision-table requirements.
QA Leader - Lê Thị Văn Anh	Own plan, framework, API automation, execution evidence, dashboard and defect lifecycle.
Tester 1 - Trương Quang Tuân	Develop unit tests, maintain workspace configuration and peer-review QA outputs.
Developer 1 - Nguyễn Đào Văn Quý	Implement/fix backend APIs, repository hooks and route defects.
Developer 2 - Lê Hữu Sơn	Implement/fix frontend login and validation behavior.

14. TEST SCHEDULE

Phase	Start	End	Owner
Test planning	01/10/2023	07/10/2023	Project Manager / QA Lead
Vitest workspace setup	08/10/2023	12/10/2023	QA Lead / Tester 1
API automation	13/10/2023	20/10/2023	QA Lead
Coverage instrumentation	21/10/2023	24/10/2023	QA Lead
Dashboard deployment	25/10/2023	27/10/2023	QA Lead
Regression and defect review	28/10/2023	01/11/2023	QA Team
v1.2 verification and closure	19/07/2026	TBD	QA Lead / Developers

15. DEFECT MANAGEMENT PROCESS

Workflow: New -> Assigned -> In Progress -> Fixed -> Retest -> Closed

Severity	Definition	Response expectation
Critical	Crash, data corruption, unauthorized access, or total loss of a core workflow.	Immediate triage; release blocker
High	Payment/booking/security rule failure with no acceptable workaround.	Fix before release
Medium	Incorrect non-critical behavior or reporting with a workaround.	Schedule and retest
Low	Cosmetic, warning, wording or minor usability issue.	Backlog unless release-critical

Each defect record must include an identifier, environment/build, summary, preconditions, reproducible steps, expected and actual results, severity, priority, owner, evidence, status, and retest result.

16. RISKS AND MITIGATION

Risk	Impact	Mitigation / contingency
Root dependency resolution mismatch	High	Install or expose application dependencies to the root Vitest workspace; lock versions; rerun all suites.

Risk	Impact	Mitigation / contingency
Deprecated workspace configuration	Medium	Migrate vitest.workspace.ts projects into test.projects in vitest.config.ts before the next major Vitest upgrade.
Timezone drift	High	Pin Asia/Ho_Chi_Minh or use controlled clocks for time-sensitive cases.
Mock divergence from production	High	Review mock contracts against repository and provider interfaces each sprint; retain a small sandbox integration set.
Incomplete API routes	Medium	Mock stable interfaces to author tests early; track blocked cases explicitly.
Misleading aggregate coverage	Medium	Report targeted-module coverage separately from full-workspace coverage and document exclusions.

17. TEST DELIVERABLES

- Approved Test Plan (this document).
- Automated unit, API integration, and UI test suites.
- JSON/HTML coverage reports and machine-readable test results.
- PCS QA Dashboard and generated dashboard data.
- Test execution, summary, automation, data, traceability, decision-table and use-case reports.
- Defect report with retest evidence.
- Postman collection and environment for supported manual checks.

18. COMMUNICATION PLAN

Activity	Frequency	Participants	Output
QA stand-up	Daily during execution	QA Lead, Tester 1	Progress, blockers, next actions
Defect triage	Twice weekly / as needed	QA, developers, PM	Severity, ownership, target fix
Status report	Weekly	QA Lead, PM, stakeholders	Execution, coverage, defects, risk
Release review	At exit gate	PM, QA Lead, relevant owners	Go/no-go recommendation

19. APPROVALS

Name	Role	Signature	Date
TRẦN QUỐC SANG	Project Manager	Pending v1.2 approval	
LÊ THỊ VĂN ANH	QA Leader	Prepared	19/07/2026
TRƯƠNG QUANG TUÂN	Tester 1 / Reviewer	Pending review	

APPENDIX A - TEST CASE TEMPLATE

Field	Required content
Test Case ID	Unique identifier, e.g. TC_API_09
Module	Functional area under test
Preconditions	Required initial state and data
Test Steps	Numbered execution actions
Expected Result	Observable acceptance condition
Actual Result	Observed result and evidence
Status	Pass / Fail / Blocked / Not Run

APPENDIX B - DEFECT REPORT TEMPLATE

Field	Required content
Defect ID	Unique identifier, e.g. DF_PAY_01
Summary	Concise defect statement
Steps to Reproduce	Repeatable sequence including data
Expected / Actual	Acceptance condition and observed behavior
Severity / Priority	Critical-High-Medium-Low / P1-P2-P3
Status / Owner	Lifecycle state and accountable person
Evidence	Logs, screenshots, response payloads or test output

APPENDIX C - DECISION TABLE SAMPLE

Registration validation example

Conditions / result	TC1	TC2	TC3	TC4	TC5	TC6	TC7	TC8
Email valid & unique	Y	Y	Y	Y	N	N	N	N
Phone valid & unique	Y	Y	N	N	Y	Y	N	N
Password strong	Y	N	Y	N	Y	N	Y	N
Expected result	Success	Err Pass	Err Phone	Err Both	Err Email	Err Both	Err Both	Err All

APPENDIX D - USE CASE TESTING SAMPLE

UC-04: Book court and pay online

Step	User / system action	Expected result
1	Select court and 09:00-10:00 slot	Slot is validated and held according to booking rules.
2	Proceed to online payment	Backend creates a PayOS payment link/QR for the pending order.
3	PayOS sends signed callback	Webhook validates the callback before accepting the update.
4	Process valid payment result	Order becomes Paid and the reserved slot is unavailable to others.

APPENDIX E - CURRENT VERIFICATION RECORD

Metric	Result
Command	npm test -- --reporter=json --outputFile=tmp-test-results.json
Executed tests	35 passed, 0 failed, 0 pending
Suite collection	37 passed; 7 failed; overall run unsuccessful
Collection blockers	Missing root-resolvable packages: @react-oauth/google, nodemailer, axios, exceljs, google-auth-library
Configuration warning	vitest.workspace.ts is deprecated; migrate to test.projects
Disposition	Resolve dependencies/configuration, rerun full suite, refresh coverage and seek approval